

Assignment 3

1. Problem Formulation

This assignment addresses the problem of generating joint-space trajectories for a two-link planar robotic arm using optimization techniques. The robot is required to move between a specified initial and final joint configuration within a fixed time duration. Unlike earlier assignments where trajectories were predefined, the joint angles at each time step are treated as optimization variables, allowing the trajectory to be computed automatically based on a performance objective.

2. Trajectory Discretization

The motion duration is divided into a finite number of equally spaced time intervals. At each time step, the joint angles of both links are defined as decision variables. This discretized representation converts the trajectory planning problem into a finite-dimensional optimization problem, where the complete joint-space motion is described by a sequence of joint configurations over time.

3. Objective Function

To generate smooth motion, the optimization minimizes the sum of squared joint accelerations across the entire trajectory. Joint accelerations are approximated using finite differences between successive time steps. By penalizing large accelerations, the objective encourages gradual changes in joint motion, resulting in smooth and physically feasible trajectories suitable for real robotic systems.

4. Constraints and Boundary Conditions

The optimization is subject to equality constraints that enforce the initial and final joint configurations. These constraints ensure that the robot begins and ends its motion at the desired poses. Additional constraints, such as joint angle limits, may also be included to reflect physical limitations of the robot, although they are optional for this assignment.

5. Numerical Optimization Approach

The resulting constrained optimization problem is solved using a numerical optimization algorithm. An initial guess, typically obtained from a linear or polynomial interpolation between the start and end configurations, is provided to the solver. The optimizer iteratively updates the joint angles to reduce the objective function while satisfying all constraints, producing an optimized joint-space trajectory.

6. Comparison with Assignment 2 Trajectories

The optimized trajectories are compared with the linear and polynomial joint-space trajectories developed in Assignment 2. Linear trajectories exhibit abrupt velocity changes at the boundaries, while polynomial trajectories improve boundary smoothness but do not minimize motion smoothness globally. The optimized trajectories demonstrate improved smoothness throughout the entire motion by explicitly minimizing acceleration, rather than relying on a fixed trajectory form.

7. Discussion and Conclusion

The results demonstrate that trajectory optimization provides a systematic and flexible approach to motion generation in joint space. By directly optimizing joint trajectories with respect to a smoothness criterion, higher-quality motion is achieved compared to interpolation-based methods. This assignment highlights the advantages of optimization-based trajectory planning and establishes a foundation for more advanced robotic motion planning and control techniques.

Python Script: Joint-Space Trajectory Optimization

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# -----
# Problem Parameters
# -----

T = 3.0          # total motion time
N = 60           # number of discretization points
dt = T / (N - 1)

q_start = np.array([-0.2, 0.3])
```

```
q_goal = np.array([1.2, 0.8])
```

```
time = np.linspace(0, T, N)
```

```
# -----
```

```
# Cost Function: Velocity + Acceleration
```

```
# -----
```

```
def trajectory_cost(z):
```

```
    q = z.reshape(N, 2)
```

```
    vel_cost = 0.0
```

```
    acc_cost = 0.0
```

```
    for k in range(N - 1):
```

```
        q_dot = (q[k+1] - q[k]) / dt
```

```
        vel_cost += np.dot(q_dot, q_dot)
```

```
    for k in range(1, N - 1):
```

```
        q_ddot = (q[k+1] - 2*q[k] + q[k-1]) / dt**2
```

```
        acc_cost += np.dot(q_ddot, q_ddot)
```

```
    return 0.3 * vel_cost + 0.7 * acc_cost
```

```
# -----
```

```
# Boundary Constraints
```

```
# -----
```

```
def constraint_start(z):
```

```
    q = z.reshape(N, 2)
```

```
return q[0] - q_start
```

```
def constraint_goal(z):
```

```
    q = z.reshape(N, 2)
```

```
    return q[-1] - q_goal
```

```
constraints = [
```

```
    {'type': 'eq', 'fun': constraint_start},
```

```
    {'type': 'eq', 'fun': constraint_goal}
```

```
]
```

```
# -----
```

```
# Initial Guess: Sinusoidal Interpolation
```

```
# -----
```

```
q_guess = np.zeros((N, 2))
```

```
for j in range(2):
```

```
    q_guess[:, j] = q_start[j] + (q_goal[j] - q_start[j]) * (
```

```
        0.5 - 0.5 * np.cos(np.pi * time / T)
```

```
    )
```

```
z0 = q_guess.flatten()
```

```
# -----
```

```
# Solve Optimization Problem
```

```
# -----
```

```
solution = minimize(
```

```
    trajectory_cost,
```

```
    z0,
```

```

    method='SLSQP',
    constraints=constraints,
    options={'maxiter': 600, 'ftol': 1e-7}
)

q_opt = solution.x.reshape(N, 2)

# -----
# Reference Polynomial Trajectory (Assignment 2)
# -----

def quintic(q0, qf, t, T):
    a0 = q0
    a1 = 0
    a2 = 0
    a3 = 10*(qf - q0)/T**3
    a4 = -15*(qf - q0)/T**4
    a5 = 6*(qf - q0)/T**5
    return a0 + a3*t**3 + a4*t**4 + a5*t**5

q_poly = np.zeros_like(q_opt)
for j in range(2):
    q_poly[:, j] = quintic(q_start[j], q_goal[j], time, T)

# -----
# Plot Results
# -----

plt.figure(figsize=(10, 5))

```

```
plt.subplot(2, 1, 1)

plt.plot(time, q_opt[:, 0], label="Optimized")
plt.plot(time, q_poly[:, 0], "--", label="Quintic Polynomial")
plt.ylabel("Joint 1 (rad)")

plt.legend()

plt.grid(True)
```

```
plt.subplot(2, 1, 2)

plt.plot(time, q_opt[:, 1], label="Optimized")
plt.plot(time, q_poly[:, 1], "--", label="Quintic Polynomial")
plt.xlabel("Time (s)")
plt.ylabel("Joint 2 (rad)")

plt.legend()

plt.grid(True)
```

```
plt.tight_layout()

plt.show()
```